



UNIVERSITI TUNKU ABDUL RAHMAN

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

JUNE 2026 TRIMESTER

UCCD3084 | Graphics Programming for Extended Reality (XR)

Project Title: Factory VR Training Simulation:

Repair, Optimization & Custom Feature

Team: DD1 Studio

Student names	IDs
Lee Chengyu	2204870
Siaw Zhen Xuan	2305656
Joelle Soo Neng Qi	2206417
Mervyn Hew Jun Jie	2104481

1.0 Project Scope

1.1 Part A

Items 3–5 addressed inconsistent baked lighting across the factory floor, box piles, and console. Fixes involved adjusting lightmap resolution and GI contribution settings on affected objects and introducing a Light Probe Group to allow the dynamic console to correctly sample nearby baked lighting.

Item 6 added a dynamic material state switch (which differentiates active and inactive states) and a particle effect during box transport to provide clear visual feedback for the crane's magnet operation. To guarantee simple, illness-free navigating, complete VR Locomotion (teleportation, ray-toggle interaction, snap turning) was also incorporated across the factory floor.

1.2 Part B

Part B: Additional Feature – Progress Tracking Display

The chosen Addition Feature by the group is the Progress Tracking Display, otherwise known as the HUD.

The scope of the part is to design and implement a Wrist-Mounted Interactive HUD & Progress Tracking System attached to the user's left VR hand. It functions like a regular HUD with all the basic requirements for the additional feature, giving real-time feedback while operating the factory crane.

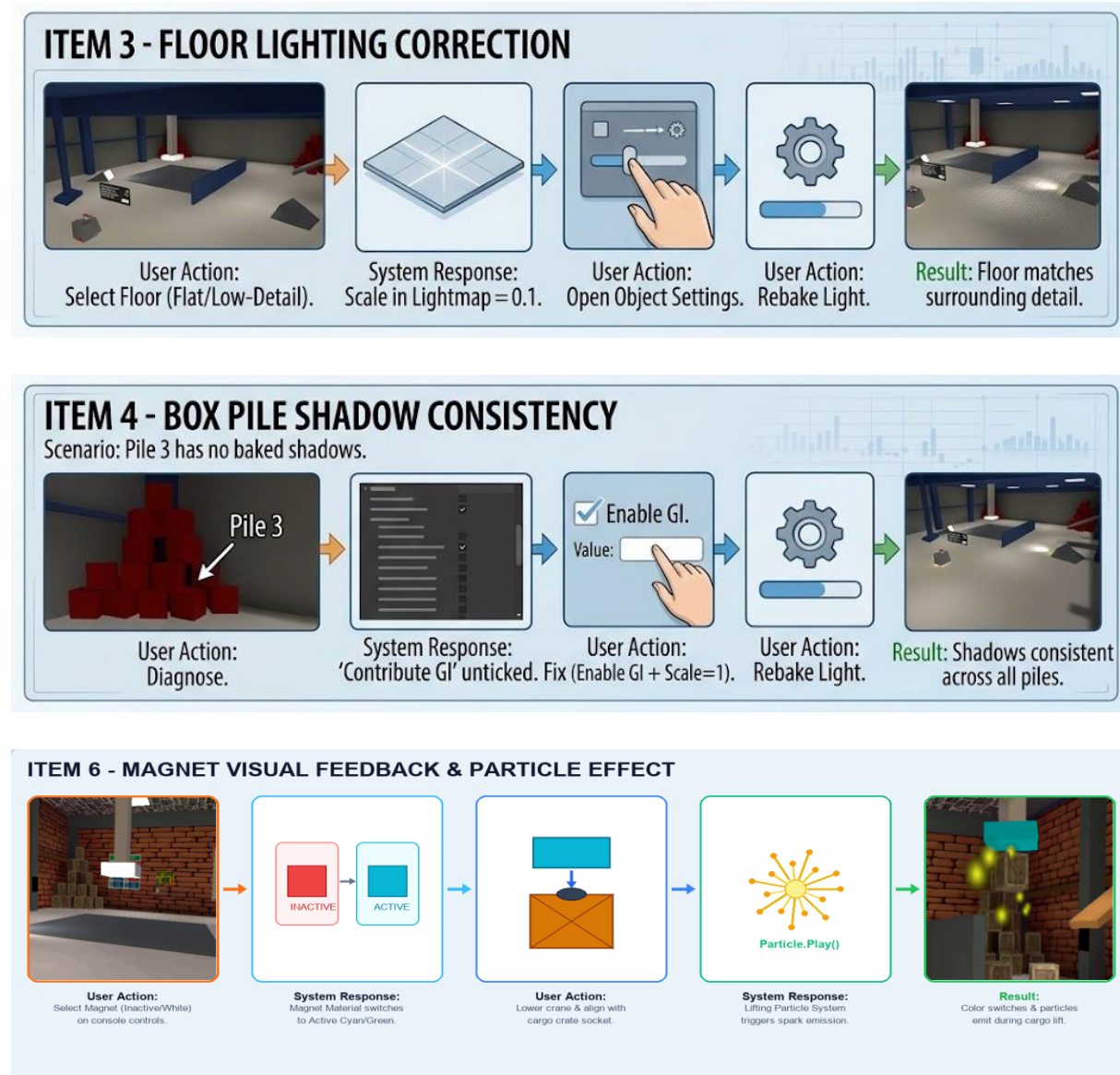
In the original prototype, trainees could not see how many boxes they had moved or how long they had taken. A wrist mounted HUD is natural in VR and common in many VR games. The user can glance at their hand at any time instead of looking away at a distant UI that may or may not be out of reach.

The user needs a way to save their progress or restart the training session without reloading the entire scene. Having quick access to comfort settings (vignette) and volume sliders ensures trainees can prevent VR motion sickness while training. All the following reasons above have led us to cramming all those problems, thus into functions, into a single HUD.

2.0 Visual Diagrams

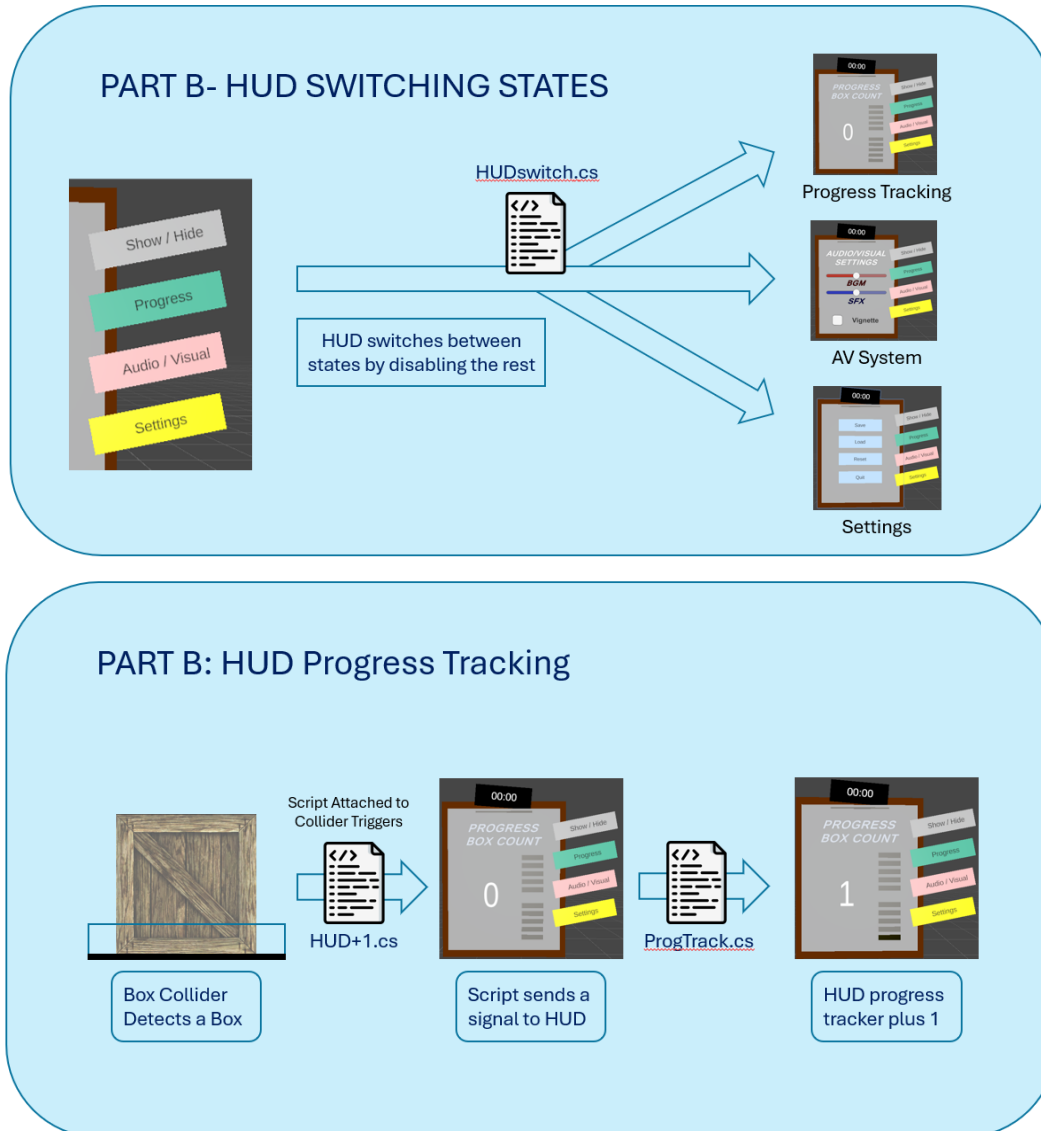
(storyboard or flowchart of the simulation — user actions vs. system responses)

2.1 Part A

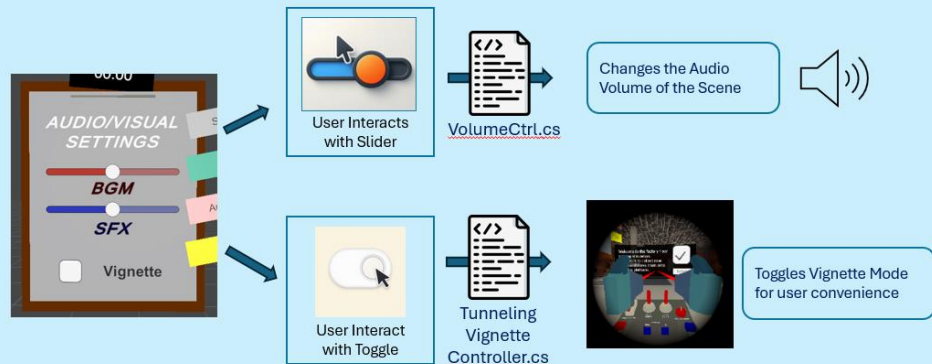


2.2 Part B.

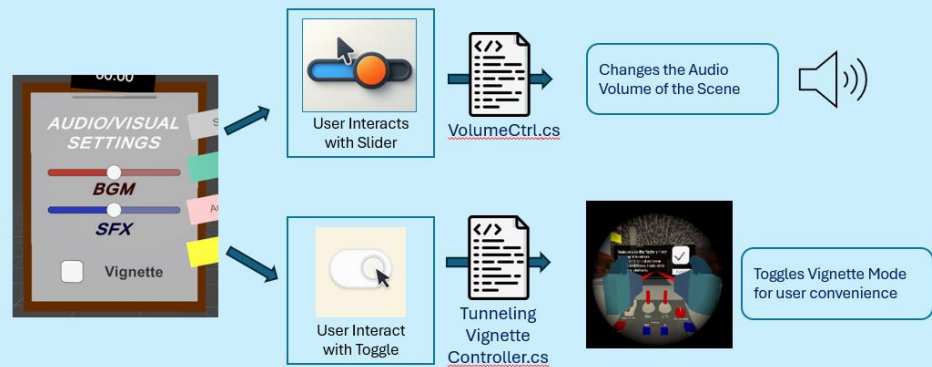
HUD Interaction & Simulation Flow

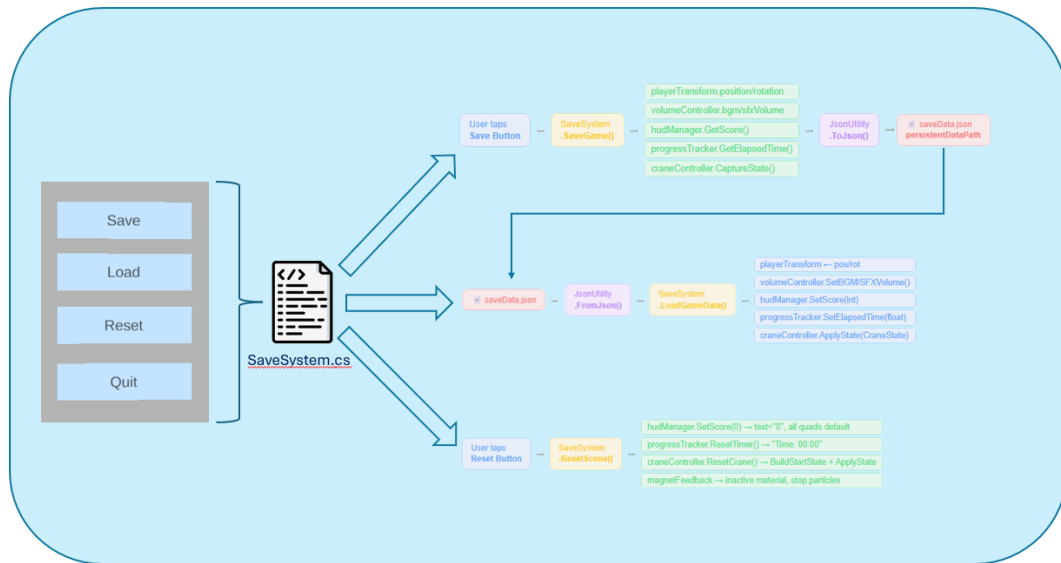


PART B: Audio / Visual



PART B: Audio / Visual

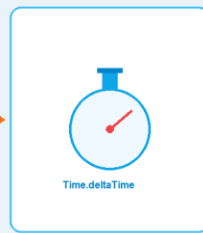




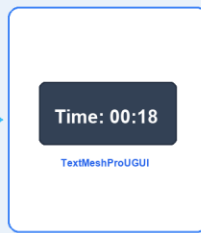
PART B - WRIST-MOUNTED HUD & REAL-TIME SESSION TIMER



User Action:
Raise left wrist to view HUD
(Initial Time: 00:00)



System Response:
ProgressTracker script counts
elapsed session time.



System Response:
Real-time UI converts seconds
to MM:SS text display.



System Action:
Save timestamp to JSON or
Reset without scene reload.



Result:
Real-time temporal feedback
displayed during training.

3.0 Repair & feature documentation

3.1 Part A

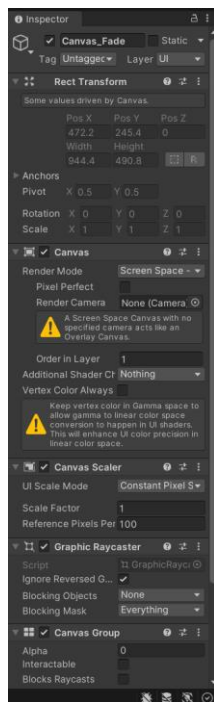
Item 1 - UI: Onboarding Fade Screen

Issue: The black fade-in screen appears positioned in the middle of the factory (at 10 meters depth) rather than covering the screen, allowing closer factory geometry to render in front of the fade.

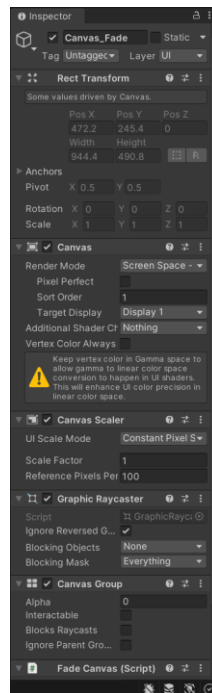
Diagnosis: Canvas_Fade was configured with Render Mode = Screen Space - Camera bound to Main Camera with Plane Distance = 10.0. Any object closer than 10 meters to the camera (the floor, console, and columns) renders in front of the canvas plane due to 3D depth testing.

Fix: Switched Canvas_Fade's Render Mode from Screen Space - Camera to Screen Space - Overlay. This renders the UI directly across the viewport without depth testing against world geometry.

Result: The fade-in renders directly on top of the viewport after all 3D geometry is drawn, guaranteeing a solid blackout that covers 100% of the user's field of view.



Before:



After:

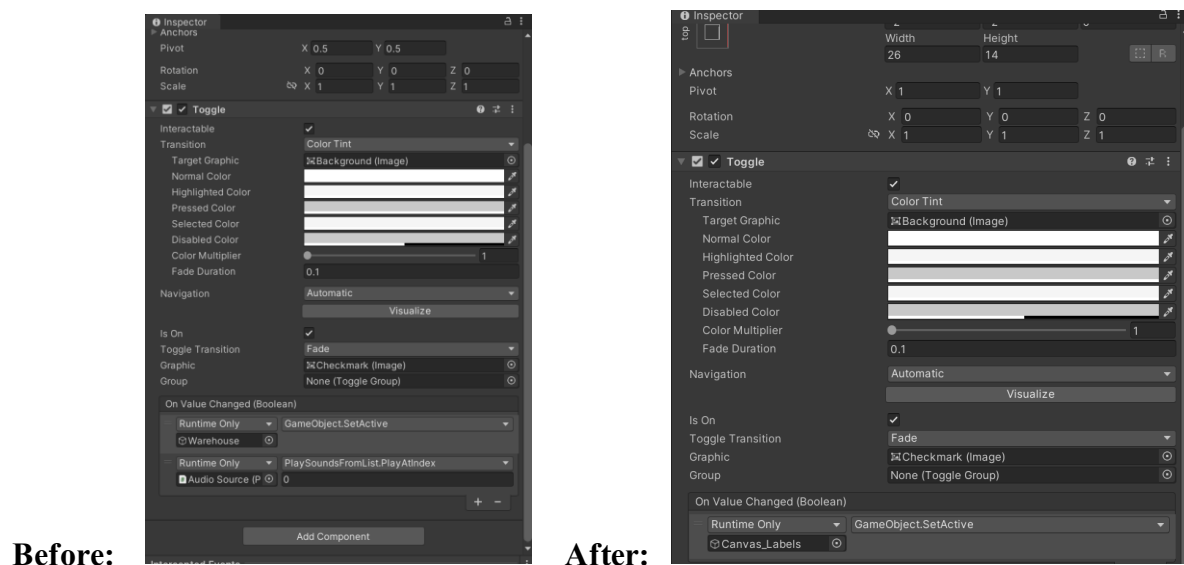
Item 2 - Toggle Logic – Console Labels

Issue: The "Console Labels" toggle on the main console made the entire warehouse environment disappear when deactivated, rather than hiding only the label text on the console surface.

Diagnosis: Inspecting the Toggle component on the Toggle_Labels GameObject revealed that its onValueChanged (Boolean) UnityEvent listener had its target object mistakenly linked to the root Warehouse GameObject (calling `GameObject.SetActive`). Consequently, unchecking the toggle deactivated the entire factory hierarchy (walls, floor, columns, lights, box piles), leaving the user standing in an empty void.

Fix: Repointed the onValueChanged persistent UnityEvent target from the root Warehouse GameObject to the Canvas_Labels GameObject (calling `GameObject.SetActive`).

Result: Deactivating the toggle now cleanly hides only the label text overlay on the console controls, leaving the physical console structure, machinery, and warehouse environment fully visible and active.



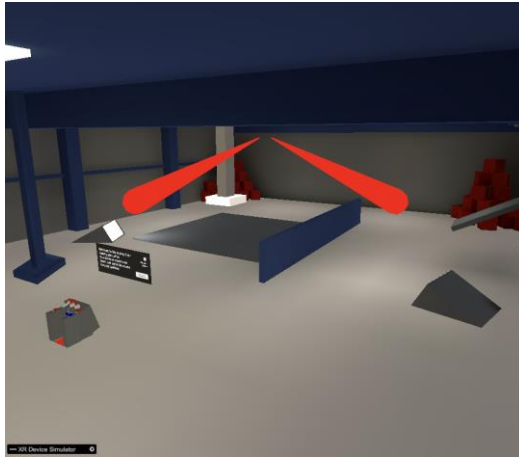
Item 3 — Lighting: Factory Floor

Issue: The factory floor appeared visually flatter and less detailed than surrounding surfaces (roof, walls, platform), lacking the same level of baked shadow/AO detail.

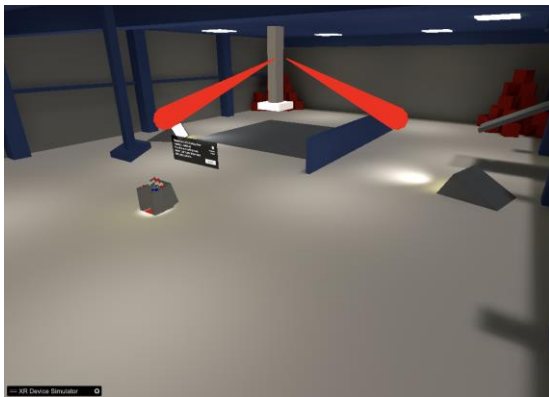
Diagnosis: Inspecting the Floor object's Mesh Renderer revealed its **Scale in Lightmap** value was set to 0.1, while comparable surfaces (Roof, Platform, Walls) were set to 1 — meaning the floor received significantly fewer lightmap texels during baking, resulting in lower-resolution, blurrier baked lighting detail.

Fix: Increased the Floor's Scale in Lightmap from 0.1 to 1 to match adjacent surfaces, then regenerated lighting (Window → Rendering → Lighting → Generate Lighting).

Result: Shadow edges and ambient occlusion detail on the floor now match the fidelity of surrounding geometry, most visibly where the floor meets shadow-casting objects like columns and box piles.



Before:



After:

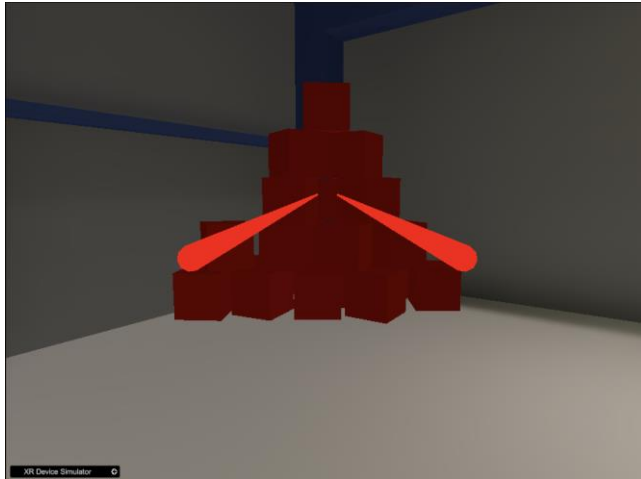
Item 4 — Lighting: Box Piles

Issue: The box pile in the back-left corner (Pile 3) lacked realistic baked shadows and ambient occlusion compared to other box piles in the scene, appearing flat and unshaded.

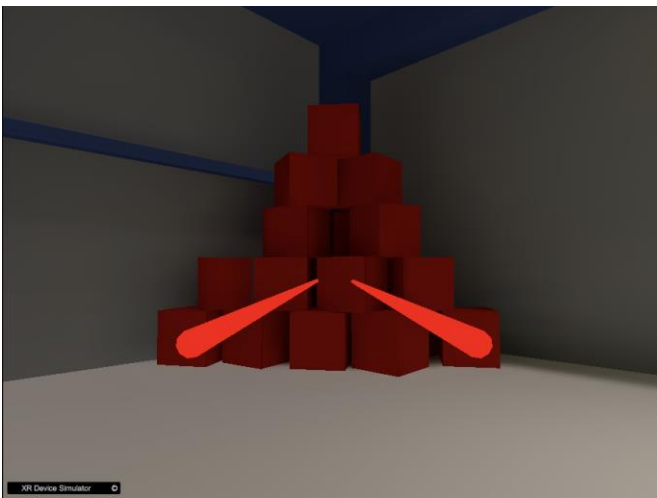
Diagnosis: Pile 3's box objects had **Contribute Global Illumination** unticked in their Static flags — meaning they were entirely excluded from the lightmap bake, unlike the other piles which had this enabled with a Scale in Lightmap of 1.

Fix: Enabled Contribute GI on Pile 3's box objects, then set Scale in Lightmap to 1 to match the other piles. Regenerated lighting.

Result: Pile 3 now shows consistent contact shadows and crevice shading matching the other box piles in the scene.



Before:



After:

Item 5 — Lighting: Console

Issue: The console did not appear lit by the fluorescent light fixture (Console_Light) directly above it, appearing dark despite the light being active.

Diagnosis: The console's interactive components (Controls, containing movable elements such as Left, Right, Center, and Change Height) are non-static, since they respond to user interaction via a TranslateObject script. Because Console_Light is a baked (not realtime) light, non-static objects near it cannot receive its lighting contribution directly — they require nearby Light Probes to sample and interpolate that baked lighting data, and no probes existed near the console.

Fix: Added a cluster of Light Probes around the Controls object using the existing Light Probe Group (Edit Light Probe Positions → Add Probe), positioned to surround the console volume.

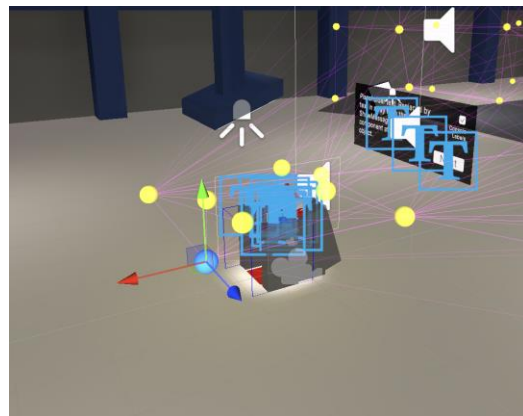
Console_Light itself was kept in Baked mode for optimization, as required. Regenerated lighting.

Result: The console now shows a visible lighting gradient and highlight definition consistent with light coming from Console_Light above it, while the light itself remains baked.

Before:



After:



Item 6: Visual Feedback: Particle Effect & Magnet State

Problem: The user could not see if the magnet was active or inactive. When a box was successfully secured and lifted by the crane, trainees had to guess whether the magnet was energised because there was no visual cue.

Diagnosis: Whether the crane magnet was on or off, the Magnet GameObject's static material stayed visibly unchanged. Moreover, the magnet's XRSocketInteractor lacked event hooks to initiate particle systems when cargo was attached or released.

Fix:

1. Created two distinct materials: Magnet_Active (emissive high-visibility green/active state) and Magnet_Inactive (standard industrial grey/inactive state).
2. Implemented a custom script MagnetVisualFeedback.cs and attached it to the Magnet prefab. The script references the magnet's MeshRenderer and a lifting ParticleSystem (sparks/magnetic field effect).

3. Hooked the magnet on/off toggle button's UnityEvent to MagnetVisualFeedback.SetMagnetActiveState(bool).
4. Wired the XRSocketInteractor component's selectEntered event to MagnetVisualFeedback.OnBoxGrabbed() to trigger particle emission when a box is picked up, and selectExited to MagnetVisualFeedback.OnBoxReleased() to stop particles when the box is deposited.
5. Configured the Particle System with Simulation Space set to Local so that particles accurately follow the magnet during crane travel.

Result: The magnet's body clearly changes from inactive grey to active green when it is powered on. An animated particle effect continuously emanates at the contact point when a box is safely locked and lifted, offering instantaneous and clear visual proof of a successful lift.

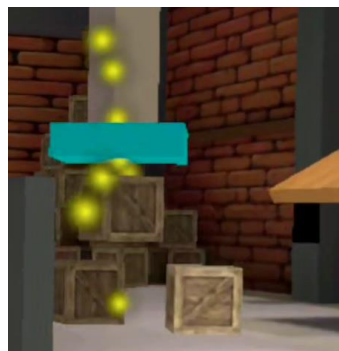


Before:



After:

Active State



Particle effect

Item 7 – Interaction Instruction Lock

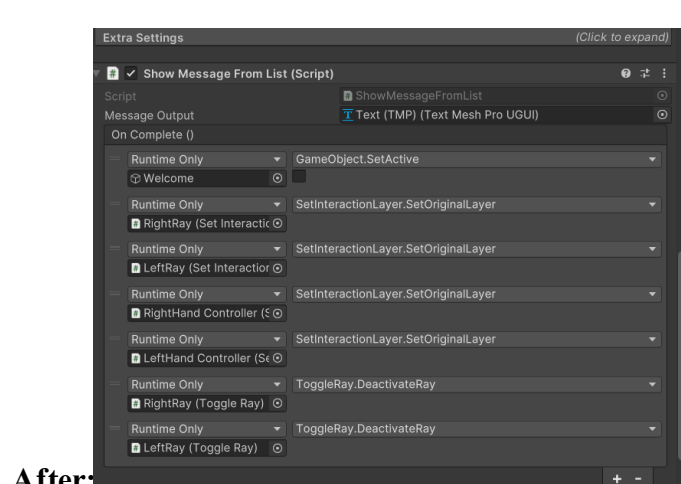
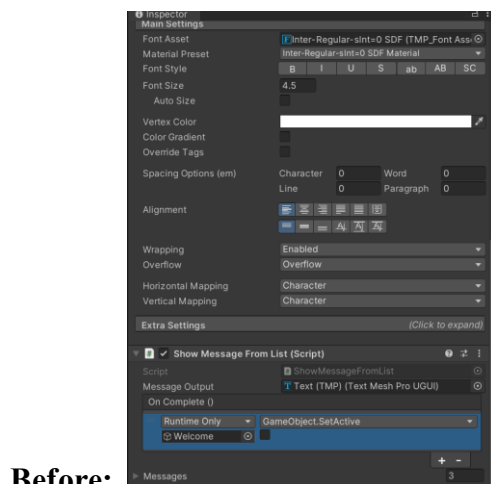
Issue: Players could grab and move console controls while onboarding instructions were still playing. Ray interactors also stayed active after the instructions finished instead of switching to direct hand grabs.

Diagnosis: All 8 console interactables (Lever, Knob, Joysticks, and Buttons) were set to the Default interaction layer with no lockout logic. The hand controllers lacked layer-switching and ray-toggling scripts, and the OnComplete () event list on the Text (TMP) ShowMessageFromList component was empty.

Fix:

1. Added a custom "Console" layer in InteractionLayerSettings and assigned the console interactables to it.
2. Attached SetInteractionLayer and ToggleRay components to the hand controllers (LeftRay, RightRay, LeftHand Controller, and RightHand Controller).
3. Configured 7 persistent event calls on Text (TMP) ShowMessageFromList.OnComplete:
 - 3.1. Call 0: Hides the instruction canvas (Welcome.SetActive(false)).
 - 3.2. Calls 1 and 2: Runs SetOriginalLayer on LeftRay and RightRay to update interaction masks.
 - 3.3. Calls 3 and 4: Runs SetOriginalLayer on LeftHand Controller and RightHand Controller so hands can grab the console controls.
 - 3.4. Calls 5 and 6: Runs DeactivateRay on LeftRay and RightRay to turn off the ray lines.

Result: Console controls stay locked while reading instructions. Once the player clicks through the final message, the instruction board disappears, the console unlocks for physical interaction, and the controller rays turn off.



Overall VR Experience: Locomotion & Comfort Navigation System

Scope & Implementation:

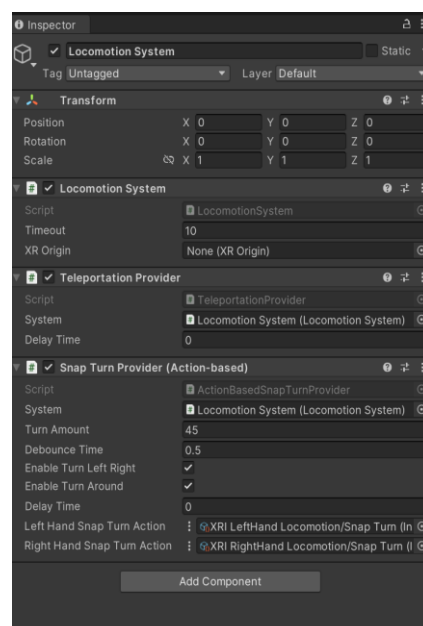
To fulfill the rubric's locomotion and comfort requirements:

1. Teleportation Locomotion: Implemented a Teleportation Provider, Teleportation Area across the factory floor, and Teleportation Anchors positioned at key training stations.
2. Interaction Switching: Configured hand controller rays to toggle via OnButtonPress (Primary 2D Axis / Grip / Button triggers), allowing seamless switching between ray-based teleportation and direct hand grabbing.
3. Comfort Settings: Integrated Snap Turn Provider for stepped rotational comfort
4. Camera & Eye-level Calibration: Set XR Origin to Device Tracking Mode with a calibrated eye-level height of 1.65m and an optimal 1.25m console offset, preventing floor-clipping and providing an authentic standing perspective.

Result: Trainees can naturally traverse the entire factory environment via smooth teleportation without experiencing nausea, and can effortlessly toggle between distant teleport navigation and close-up console operation.



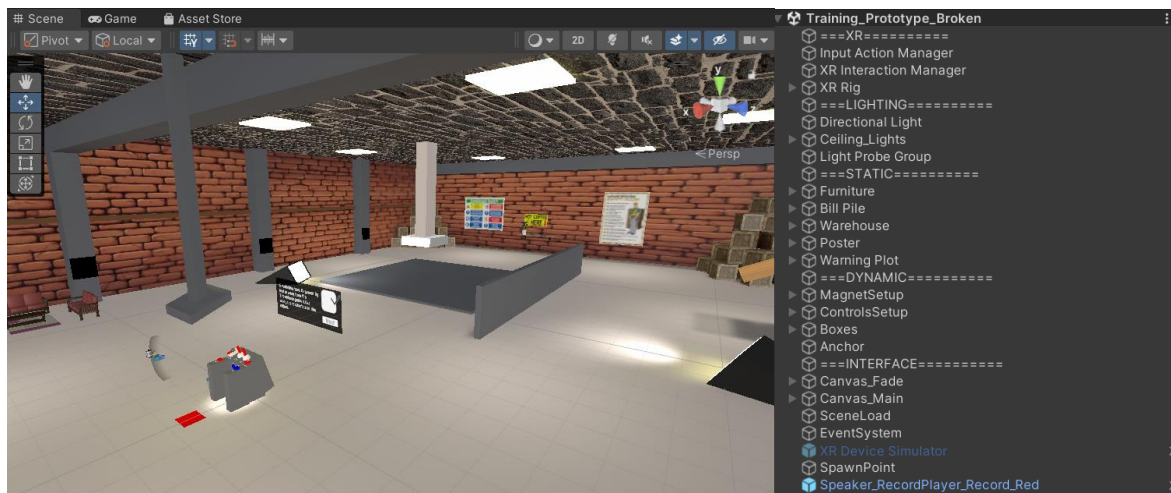
Teleport



Inspector

3.2 Part B

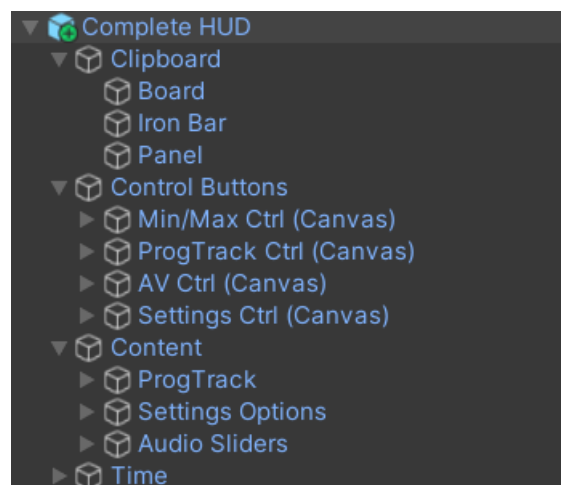
A) System Overview



The Training Prototype Broken scene is a VR crane-operating training simulation set in an industrial warehouse. The 'intern' aka the user operates a gantry crane from a physical console to pick up and deliver cargo boxes. The HUD (Progress Tracking Display) is a 3D clipboard attached to the player's view, providing real-time feedback on training performance, session management, and comfort settings.

B) HUD Prefab Structure

The HUD is a single prefab called Complete HUD — a 3D clipboard model with a Unity UI Canvas overlaid on it. Six scripts are attached to the root GameObject, and the UI is organized into three switchable tab panels.



Scripts on Root:

Script	Class Name	Purpose
ProgTrack.cs	HUDManager	Tracks delivery score and updates the score text + 10-segment light meter
HUDSwitch.cs	ViewSwitcher	Tab navigation — switches between Progress, Settings, and Audio panels
Minimize.cs	ClipboardMinimize	Animated minimize/expand with smooth Lerp scaling
SaveSystem.cs	SaveSystem	JSON-based save/load, in-memory scene reset, and application quit
VolumeCtrl.cs	VolumeController	Controls BGM and SFX audio volume via sliders
Vignette.cs	ManualVignetteToggle	Toggle / Disable Vignette mode

C) UI Element #1 Score Display & Light Meter (Progress Tab)

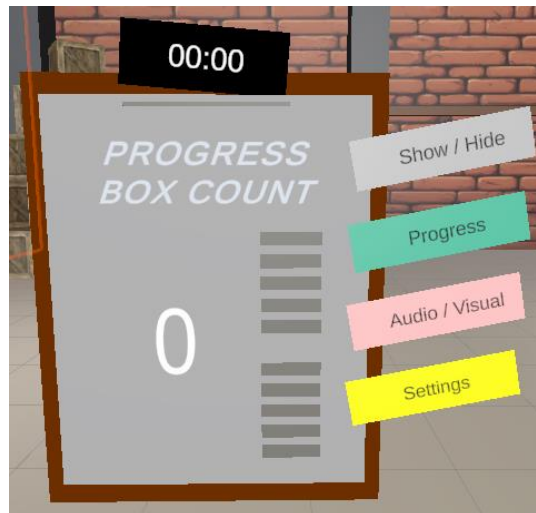
Script: ProgTrack.cs class HUDManager

The Progress tab shows two visual elements:

- Score counter: A TextMeshPro (TMP) text element displaying the current delivery count as a number
- 10-segment light meter: Ten Quad renderers that light up progressively. Each successful delivery lights one more segment (yellow material). The meter cycles every 10 deliveries.

How it works:

- When a box enters the delivery zone, HUD+1.cs (ScoreIncrementTrigger) calls HUDManager.IncrementScore()
- IncrementScore() increments the internal score counter and calls UpdateDisplay()
- UpdateDisplay() sets the TMP text to the score value and calculates how many quads to light: $\text{NoLight} = ((\text{score} - 1) \% 10) + 1$
- Quads 0 through NoLight-1 get the yellowMat (lit), the rest get defaultMat (unlit)

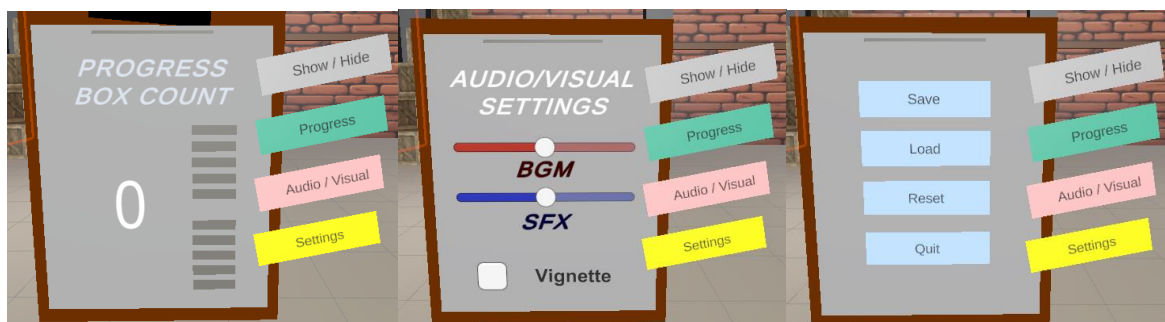


D) UI Element #2 — Tab Navigation System

Script: HUDSwitch.cs — class ViewSwitcher

The HUD clipboard has three tab buttons at the top that switch between panels. Only one panel is visible at a time — the others are deactivated via `SetActive(false)`.

Tab	Panel Shown	Contents
Progress	progressPanel	Score counter, 10-segment light meter
Settings	settingsPanel	Save, Load, Reset, Quit buttons
Audio	audioPanel	BGM slider, SFX slider, Vignette toggle



E) UI Element #3 — Session Timer

Feature Overview:

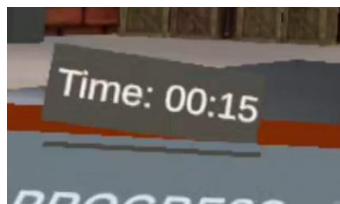
As part of the Part B Progress Tracking Display, a real-time Stopwatch / Session Timer was designed and integrated at the top of the wrist-mounted HUD. It provides trainees with

immediate temporal feedback, measuring the efficiency of their factory repair and crane operation tasks.

Technical Implementation:

1. UI Layout: Created a dedicated header banner displaying the session time in a standardized "Time: MM:SS" format using TextMeshPro for crisp in-headset legibility.
2. Core Logic (ProgressTracker.cs):
 - In each frame Update(), elapsedTime accumulates Time.deltaTime when active.
 - The elapsed seconds are converted into minutes and seconds via $\text{Mathf.FloorToInt}(\text{elapsedTime} / 60F)$ and formatted into strings dynamically.

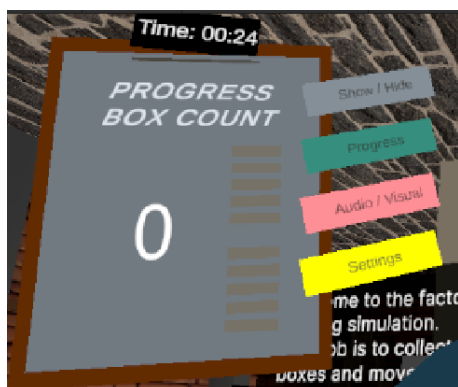
Result: The stopwatch runs smoothly with zero overhead, giving trainees constant awareness of their operational performance right on their wrist. It persists across game saves and supports non-destructive resets, fulfilling all core rubric requirements.



F) UI Element #4 — Minimize / Expand Animation

Script: Minimize.cs — class ClipboardMinimize

The Minimize button toggles the HUD between full size and a compact docked state. It uses a coroutine to smoothly lerp the clipboard's localScale from (1,1,1) down to (0.2, 0.2, 0.2) and reposition it. All child panel objects are hidden during minimization and restored (with cached active states) on expand.



G) UI Element #5 — Save / Load / Reset / Quit System

Script: SaveSystem.cs — class SaveSystem (Assisted by AI)

The Settings tab exposes four session management buttons. The SaveSystem serializes the full game state to a JSON file and can restore it, reset the training scenario to its starting state, or exit the application.

Button	Method	What It Does
Save	SaveGame()	Serializes player position, volumes, score, timer, and full crane/box state → saveData.json
Load	LoadGame()	Reads saveData.json and restores all the above state
Reset	ResetScene()	Returns score to 0, timer to 00:00, crane to start position, destroys spawned boxes, restores original boxes — without reloading the scene
Quit	QuitGame()	Exits the application (or stops Play Mode in Editor)

Data saved in GameState:

playerPosition, playerRotation, bgmVolume, sfxVolume, score, elapsedTime, craneState.



H)

UI Element #6 — Audio Volume Controls

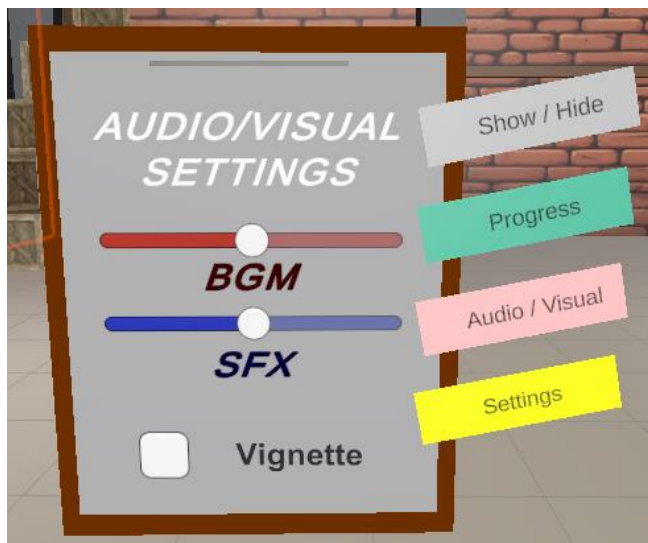
UI Element #7 — Vignette Comfort Toggle

Script: VolumeCtrl.cs — class VolumeController

Two UI Sliders on the Audio/Visual panel control background music and sound effects independently.

Script: Vignette.cs — class ManualVignetteToggle

A UI Toggle on the Audio/Visual panel enables or disables the XR Interaction Toolkit's tunneling vignette, a comfort feature that reduces motion sickness by adding a peripheral darkening effect during movement.



4.0 Team Contributions

Member	Responsibility	Key Contributions
Lee Chengyu	Part B HUD, Visual and Audio Design	Implemented Part B HUD (Progress Tracking, AV settings, Save System & Reset), implemented textures on various assets & background audio.
Joelle Soo Neng Qi	Lighting & Baking	Fixed Item 3,4,5 — floor lightmap resolution, box pile GI/shadow consistency, console lighting via Light Probe Group
Siaw Zhen Xuan	Visual Feedback, Locomotion & Session Timer	Implemented Part A Item 6 (Magnet color change & lifting particle effect), integrated full VR Locomotion (Teleportation, Ray toggling, Snap Turn), and implemented the Part B HUD Stopwatch / Session Timer
Mervyn Hew Jun Jie	UI Systems and Interaction Gating	Repaired Items 1, 2 and 7 (fade screen overlay, console labels scope, and instruction lock).